

TASK 02 — Database Indexing, Query Optimization & Performance

Step 1 — Understand indexes

An **index** in a database is a data structure that helps the database **find records faster**.

Think of a database table like a **large book**.

- Without an index → database may scan many rows one by one.
- With an index → database can quickly locate the required rows.

1. What is an Index:

Suppose we have an orders table:

	id	user_id	order_status	created_at
1	101		PAID	2026-09-01
2	205		SHIPPED	2026-09-02
3	101		PAID	2026-09-03
...	...		...	...

If we frequently execute:

```
SELECT *
```

```
FROM orders
```

```
WHERE user_id = 101;
```

An index on `user_id` can help the database find those orders much faster.

```
CREATE INDEX idx_orders_user_id
```

```
ON orders(user_id);
```

Simple idea

Without Index

Database



Check row 1

↓

Check row 2

↓

Check row 3

↓

...

↓

Find user_id = 101

With Index

Database

↓

Index on user_id

↓

Find 101 quickly

↓

Get matching orders

2. Primary-Key Index

A **primary key** uniquely identifies every row.

Example:

```
CREATE TABLE orders (  
    id BIGINT PRIMARY KEY,  
    user_id BIGINT,  
    order_status VARCHAR(20),  
    created_at TIMESTAMP  
);
```

Here:

id = 1

id = 2

id = 3

Each id must be unique.

Most relational databases automatically create an index associated with the primary key.

So this query can be very efficient:

```
SELECT *
```

```
FROM orders
```

```
WHERE id = 5000;
```

Important

Primary key:

- Identifies a row uniquely.
- Cannot normally contain duplicate values.
- Usually has an associated index.
- Is commonly used for fast lookups.

3. Unique Index

A **unique index** prevents duplicate values in a column or combination of columns.

For example, suppose every order has a unique external order number:

```
CREATE UNIQUE INDEX idx_orders_order_number
```

```
ON orders(order_number);
```

Now:

ORD1001 

ORD1002 

ORD1003 

ORD1001  duplicate

The database will reject duplicate order_number values.

Example

```
CREATE TABLE users (  
    id BIGINT PRIMARY KEY,  
    email VARCHAR(255)  
);  
  
CREATE UNIQUE INDEX idx_users_email  
ON users(email);
```

This prevents two users from having the same email address.

4. Composite Index

A **composite index** is an index created using **multiple columns**.

Example:

```
CREATE INDEX idx_orders_user_status  
ON orders(user_id, order_status);
```

This index contains:

user_id + order_status

It is useful for queries such as:

```
SELECT *  
  
FROM orders  
  
WHERE user_id = 101  
  
AND order_status = 'PAID';
```

Important concept: column order matters

Consider:

```
CREATE INDEX idx_orders_user_status  
ON orders(user_id, order_status);
```

The database can efficiently use this index for:

WHERE user_id = 101

and:

WHERE user_id = 101

AND order_status = 'PAID'

But it may not be equally useful for:

WHERE order_status = 'PAID'

because user_id is the **first column** in the index.

Think of it like:

(user_id, order_status)



First search by

user_id



Then search by

order_status

5. When Do Indexes Help?

Indexes are especially useful when:

Frequently searching

SELECT *

FROM orders

WHERE user_id = 101;

Filtering

SELECT *

FROM orders

WHERE order_status = 'PAID';

Sorting

```
SELECT *  
FROM orders  
ORDER BY created_at DESC;
```

Joining tables

```
SELECT *  
FROM orders o  
JOIN users u  
ON o.user_id = u.id;
```

Range queries

```
SELECT *  
FROM orders  
WHERE created_at >= '2026-09-01'  
AND created_at < '2026-10-01';
```

Indexes can significantly reduce the amount of data the database needs to examine.

6. When Do Too Many Indexes Become a Problem?

Indexes improve **read/search performance**, but they are not free.

Suppose your table has:

orders

├— id

├— user_id

├— order_status

├— created_at

├— total_amount

You might create:

Index 1 → id

Index 2 → user_id

Index 3 → order_status

Index 4 → created_at

Index 5 → total_amount

Index 6 → user_id + order_status

Index 7 → user_id + created_at

Having too many indexes can cause problems.

Problem 1 — INSERT becomes slower

When inserting:

```
INSERT INTO orders (...);
```

the database must update the relevant indexes too.

INSERT row

↓

Update table

↓

Update Index 1

↓

Update Index 2

↓

Update Index 3

↓

...

More indexes can mean more work.

Problem 2 — UPDATE becomes slower

If an indexed column changes:

```
UPDATE orders
```

```
SET order_status = 'SHIPPED'
```

WHERE id = 100;

the corresponding index must also be updated.

Problem 3 — DELETE becomes slower

When deleting a row:

DELETE FROM orders

WHERE id = 100;

the database also needs to remove the relevant index entries.

Problem 4 — Extra storage

Indexes consume disk/storage space.

Step 2 — Create Sample Data

The next step is to create **realistic test data**.

10,000+ orders

Why:

Because testing with only 10 or 20 rows may not show meaningful performance differences.

For example:

100 rows

↓

Query = very fast

10,000 rows

↓

Difference becomes easier to observe

1,000,000 rows

↓

Index impact can become much more noticeable

The goal is to create enough data to compare:

Query WITHOUT index

VS

Query WITH index

Example orders table

```
CREATE TABLE orders (  
    id BIGINT PRIMARY KEY,  
    user_id BIGINT,  
    order_status VARCHAR(20),  
    total_amount DECIMAL(10,2),  
    created_at TIMESTAMP  
);
```

You can then populate it with thousands of records.

For example:

id → 1 to 10000

user_id → different users

status → PAID / PENDING / SHIPPED / CANCELLED

amount → different order amounts

date → different dates

Step 3 — Identify Frequently Searched Fields

Before creating an index, identify **which columns your application searches frequently**.

For example, your application may frequently execute:

Query 1

```
SELECT *
```

```
FROM orders
```

```
WHERE user_id = 101;
```

So:

user_id

may need an index.

Query 2

```
SELECT *
```

```
FROM orders
```

```
WHERE order_status = 'PAID';
```

So:

order_status

may be considered for indexing.

Query 3

```
SELECT *
```

```
FROM orders
```

```
WHERE created_at >= '2026-09-01';
```

So:

created_at

may be considered for indexing.

Overall Process

The complete idea from the image is:

Step 1

Understand indexes



Step 2

Create 10,000+ sample orders



Step 3

Identify frequently searched columns



Step 4

Run queries WITHOUT indexes



Measure performance



Step 5

Create appropriate indexes



Step 6

Run the same queries again



Compare performance

Example

Without index:

```
SELECT *
```

```
FROM orders
```

```
WHERE user_id = 101;
```

Then create:

```
CREATE INDEX idx_orders_user_id
```

```
ON orders(user_id);
```

Run the same query again:

```
SELECT *
```

```
FROM orders
```

```
WHERE user_id = 101;
```

Step 4 — Run Queries Without Additional Indexes

```
SELECT *  
  
FROM orders  
  
WHERE user_id = ?;
```

Here, ? is a **parameter placeholder**. Your application will provide the actual user_id.

Example:

```
SELECT *  
  
FROM orders  
  
WHERE user_id = 101;
```

What happens without an index?

If there is no index on user_id, the database may need to check many rows in the orders table.

orders table



Check row 1



Check row 2



Check row 3



...



Find user_id = 101

This is commonly called a **table scan** or **sequential scan**, depending on the database.

With a large table, this can require significant work.

Measure Query Performance

We don't just run the query—we want to know **how the database executed it**.

A useful tool is the **execution/query plan**.

Example, in PostgreSQL:

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM orders
```

```
WHERE user_id = 101;
```

The database can show information such as:

Execution Time

Rows examined

Rows returned

Scan type

Estimated cost

Actual execution

The important thing is to record the result **before adding the index**.

Step 5 — Add Indexes

Now we know that `user_id` is frequently searched.

So we can create an index:

```
CREATE INDEX idx_orders_user_id
```

```
ON orders(user_id);
```

What does this do?

It creates a separate data structure that helps the database locate rows based on `user_id`.

Before:

Orders

|— Row 1

|— Row 2

|— Row 3

|— ...

└— Row 10,000

Database may scan many rows

After:

Orders

↑

|

Index on user_id

|

↓

Find user_id = 101

|

↓

Matching orders

The database can potentially avoid examining every row.

Step 6 — Test Again

EXPLAIN ANALYZE

SELECT *

FROM orders

WHERE user_id = 101;

Use the same query before and after the index so that the comparison is fair.

Before vs After

Before Index

Query

↓

Search orders table

↓

Possibly scan many rows

↓

Return matching orders

Example:

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM orders
```

```
WHERE user_id = 101;
```

Record the execution information.

Create Index

```
CREATE INDEX idx_orders_user_id
```

```
ON orders(user_id);
```

After Index

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM orders
```

```
WHERE user_id = 101;
```

Now compare the execution plan.

BEFORE INDEX



Query Performance



Record Result



CREATE INDEX



AFTER INDEX



Query Performance



Compare Results

Simple Real-World Example

We have **10,000 orders**.

You want to find all orders belonging to user 101.

Without index

```
SELECT *  
FROM orders  
WHERE user_id = 101;
```

The database may have to examine a large portion of the table.

With index

```
CREATE INDEX idx_orders_user_id  
ON orders(user_id);
```

Then:

```
SELECT *  
FROM orders  
WHERE user_id = 101;
```

The database can potentially use the index to locate matching rows more efficiently.

What Should You Compare

After running the query before and after the index, compare:

Metric	Before Index	After Index
Execution time	Record	Record
Scan type	Record	Record
Rows examined	Record	Record
Rows returned	Record	Record
Query cost	Record	Record

Step 7 — Study Composite Indexes

A **composite index** is an index created using **two or more columns**.

For an orders table, we can create it like this:

```
CREATE INDEX idx_orders_user_created
```

```
ON orders(user_id, created_at);
```

This means the database creates one index based on:

user_id



created_at

Why do we need a composite index

Suppose your application frequently runs this query:

```
SELECT *
```

```
FROM orders
```

```
WHERE user_id = 101
```

```
ORDER BY created_at DESC;
```

The query is doing two things:

1. Finding orders for a particular user_id
2. Ordering those orders by created_at

A composite index on:

(user_id, created_at)

can be useful for this type of query.

Simple example

Suppose the data is:

user_id	created_at	order_status
---------	------------	--------------

101	2026-09-01	PAID
-----	------------	------

101	2026-09-05	SHIPPED
-----	------------	---------

101	2026-09-10	PAID
-----	------------	------

102	2026-09-02	PAID
-----	------------	------

102	2026-09-08	SHIPPED
-----	------------	---------

The index is organized primarily by user_id, and then by created_at.

Index

```
└─ user_id = 101
  │
  │   └─ created_at = Sep 01
  │   │
  │   └─ created_at = Sep 05
  │   │
  │   └─ created_at = Sep 10
  │
  └─ user_id = 102
    │
    │   └─ created_at = Sep 02
    │   │
    │   └─ created_at = Sep 08
```

Why Does Column Order Matter

Consider:

```
CREATE INDEX idx_orders_user_created
ON orders(user_id, created_at);
```

The order is:

1st → user_id

2nd → created_at

Query using both columns

```
SELECT *
FROM orders
WHERE user_id = 101
AND created_at >= '2026-09-01';
```

The index can be very useful.

Query using only the first column

```
SELECT *
FROM orders
WHERE user_id = 101;
```

The index can also be useful because user_id is the first column.

Query using only the second column

```
SELECT *
```

```
FROM orders
```

```
WHERE created_at >= '2026-09-01';
```

The (user_id, created_at) index may not be as useful because created_at is not the leading column.

Easy rule

```
INDEX(user_id, created_at)
```

↓

First column

↓

user_id

↓

Second column

↓

created_at

Think of it like a person's address:

Country → State → City → Street

You generally organize the search from the earlier level to the later level.

Step 8 — Study Query Optimization

Query optimization means writing and executing database queries in a way that avoids unnecessary work and improves performance.

1. Avoid unnecessary SELECT *

This query:

```
SELECT *
```

```
FROM orders
```

```
WHERE user_id = 101;
```

requests **every column**.

Suppose the table contains:

id
user_id
order_status
total_amount
created_at
shipping_address
payment_details
...

But maybe you only need:

```
SELECT id, order_status, total_amount  
FROM orders  
WHERE user_id = 101;
```

This is better when you don't need the other columns because the database and application don't have to process unnecessary data.

Simple rule

Avoid when unnecessary:

```
SELECT *
```

Request only what you need:

```
SELECT id, order_status, total_amount
```

2. Pagination

your database contains:

1,000,000 orders

You should not normally send all 1,000,000 records to the application at once.

Instead, use **pagination**.

For example:

Page 1 → 20 orders

Page 2 → 20 orders

Page 3 → 20 orders

...

A simple example:

```
SELECT id, user_id, order_status  
FROM orders  
ORDER BY id  
LIMIT 20 OFFSET 0;
```

Next page:

```
SELECT id, user_id, order_status  
FROM orders  
ORDER BY id  
LIMIT 20 OFFSET 20;
```

Pagination reduces the amount of data returned to the application at one time.

For very large tables, **keyset/cursor pagination** can be preferable to large OFFSET values.

3. Filtering at Database Level

Suppose you want only paid orders.

Less efficient approach

Retrieve many orders and filter them in Java/Python/application code.

Database



Send many orders



Application



Filter PAID orders

Better approach

Let the database filter them:

```
SELECT *  
FROM orders  
WHERE order_status = 'PAID';
```

Now:

Database



Filter PAID orders



Return required records



Application

The database is designed to perform filtering efficiently.

4. Avoiding N+1 Queries

The **N+1 query problem** commonly happens when an application first retrieves a list and then performs another query for each item.

For example:

Query 1 → Get 100 orders

Then:

Query 2 → Get user for order 1

Query 3 → Get user for order 2

Query 4 → Get user for order 3

...

Query 101 → Get user for order 100

Total:

1 + 100 = 101 queries

This is called an **N+1 query problem**.

```
SELECT o.id,  
       o.order_status,  
       u.name  
FROM orders o  
JOIN users u  
ON o.user_id = u.id;
```

Now the database can retrieve the related information in a single query rather than repeatedly querying for each row.

5. Proper Joins

A **JOIN** combines related data from different tables.

For example:

users

| user_id

orders

Query:

SELECT

o.id,

o.order_status,

u.name

FROM orders o

JOIN users u

ON o.user_id = u.id;

The important part is:

ON o.user_id = u.id

This tells the database how the two tables are related.

Proper joins can avoid unnecessary queries and help retrieve related information efficiently.

6. Query Plans

A **query plan** tells you how the database intends to execute a query.

Query

↓

Index Scan

↓

Filter

↓

Return rows

or:

Query



Sequential/Table Scan



Check many rows



Return rows

You can inspect a plan using commands such as:

PostgreSQL

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM orders
```

```
WHERE user_id = 101;
```

MySQL

```
EXPLAIN
```

```
SELECT *
```

```
FROM orders
```

```
WHERE user_id = 101;
```

The exact commands and output differ between database systems.

Step 9 — Use Database Execution Plans

This means we should **look inside the database's execution plan** rather than simply assuming that a query is fast.

Example:

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM orders
```

```
WHERE user_id = 101;
```

The database may tell you things such as:

Scan method

Rows estimated

Rows actually returned

Execution time

Index used

Cost

You can then determine whether your index is actually helping.

Step 10 — Create Performance Report

The purpose is to record **what you tested, what changed, and whether the change improved performance.**

1. Query Tested

First, document the SQL query you tested.

For example:

```
SELECT *
```

```
FROM orders
```

```
WHERE user_id = 101;
```

This tells anyone reading the report exactly **which query was evaluated.**

2. Before Performance

Next, record how the query performed **before adding the new index.**

For example:

Before Index:

Execution time: 25 ms

Scan type: Sequential Scan

Rows examined: 10,000

Rows returned: 50

The exact numbers will depend on your database and hardware.

You can obtain this information using a query plan.

PostgreSQL

```
EXPLAIN ANALYZE
```

```
SELECT *  
FROM orders  
WHERE user_id = 101;
```

MySQL

```
EXPLAIN  
SELECT *  
FROM orders  
WHERE user_id = 101;
```

3. Index Added

Document the index that you created.

For example:

```
CREATE INDEX idx_orders_user_id  
ON orders(user_id);
```

Your report should clearly state:

Index added:

idx_orders_user_id

Column:

user_id

If you created a composite index, document that as well:

```
CREATE INDEX idx_orders_user_created  
ON orders(user_id, created_at);
```

4. After Performance

Now run the **same query again** after creating the index.

```
EXPLAIN ANALYZE
```

```
SELECT *  
FROM orders  
WHERE user_id = 101;
```

Record the new results.

For example:

After Index:

Execution time: 5 ms

Scan type: Index Scan

Rows returned: 50

Again, these numbers are only examples—you should use the actual results from your database.

5. Reason for the Change

Finally, explain **why the performance changed**.

For example:

The `user_id` column is frequently used in the WHERE clause. An index was created on `user_id`, allowing the database to locate matching orders more efficiently instead of scanning a large portion of the table.

You should avoid simply writing:

"The index made it faster."

Instead, explain **what changed in the execution plan**.

For example:

Before:

Sequential/Table Scan



Many rows examined

After:

Index-based access



Matching rows located more efficiently

Example Performance Report

You could create your report like this:

Database Query Performance Report

1. Query Tested

```
SELECT *  
FROM orders  
WHERE user_id = 101;
```

2. Before Index

Index: None on user_id
Execution Time: 25 ms
Scan Type: Sequential Scan

3. Index Added

```
CREATE INDEX idx_orders_user_id  
ON orders(user_id);
```

4. After Index

Index: idx_orders_user_id
Execution Time: 5 ms
Scan Type: Index Scan

5. Reason for Change

The query frequently searches orders using user_id.
The index provides an efficient lookup structure for user_id,
so the database can locate matching records without scanning
the entire table in the same way as before.